Programação Orientada a Objetos

Herança – Parte 1

Professor: Pedro Montoaneli
pedromontoaneli@teiacoltec.org

Sumário

- Introdução
  - Relação “é um”
  - Motivação: Códigos repetidos
  - Sintaxe da Herança
  - O especificador `protected`
- Exemplo
- Diagramas UML
- Exercícios

Relação “é um”

- Na composição, a classe A **contém** a classe B.
- Na herança, a classe A **é uma** classe B.



Motivação: Códigos repetidos

- Nas classes Aluno e Professor, os atributos `nome` e `idade`, e o método `mensagem()` são repetidos na declaração.
- Caso seja necessário modificar o método `mensagem()`, é necessário repeti-la nas duas (ou mais) classes.
- Seria útil subir os atributos comuns para uma classe mais geral.

```
class Professor {
private:
string nome, titulacao;
int idade;
public:
void mensagem() const { }
};
class Aluno {
private:
string nome;
int idade;
int matrícula;
public:
void mensagem() const { }
};
```

Sintaxe da Herança

- Os dois pontos : determinam a herança. O tipo da herança é `public` e a classe herdada é `Pessoa`.
- A classe derivada (`Professor/Aluno`) já possui os atributos `nome`, `idade` e `apresentar`.

```
class Pessoa {
protected:
string nome;
int idade;
public:
void apresentar() const { }
};
class Aluno : public Pessoa {};
class Professor : public Pessoa {};
```

O especificador `protected`

- Atributos `private` de uma classe base não podem ser acessados pela classe derivada.
- Atributos `public` de uma classe base podem ser acessados pela classe derivada, mas também de forma livre pelo usuário.
- O atributo `protected` surge como um encapsulamento intermediário. Ele pode ser acessado pela classe derivada, mas não pode ser acessado pelo usuário.

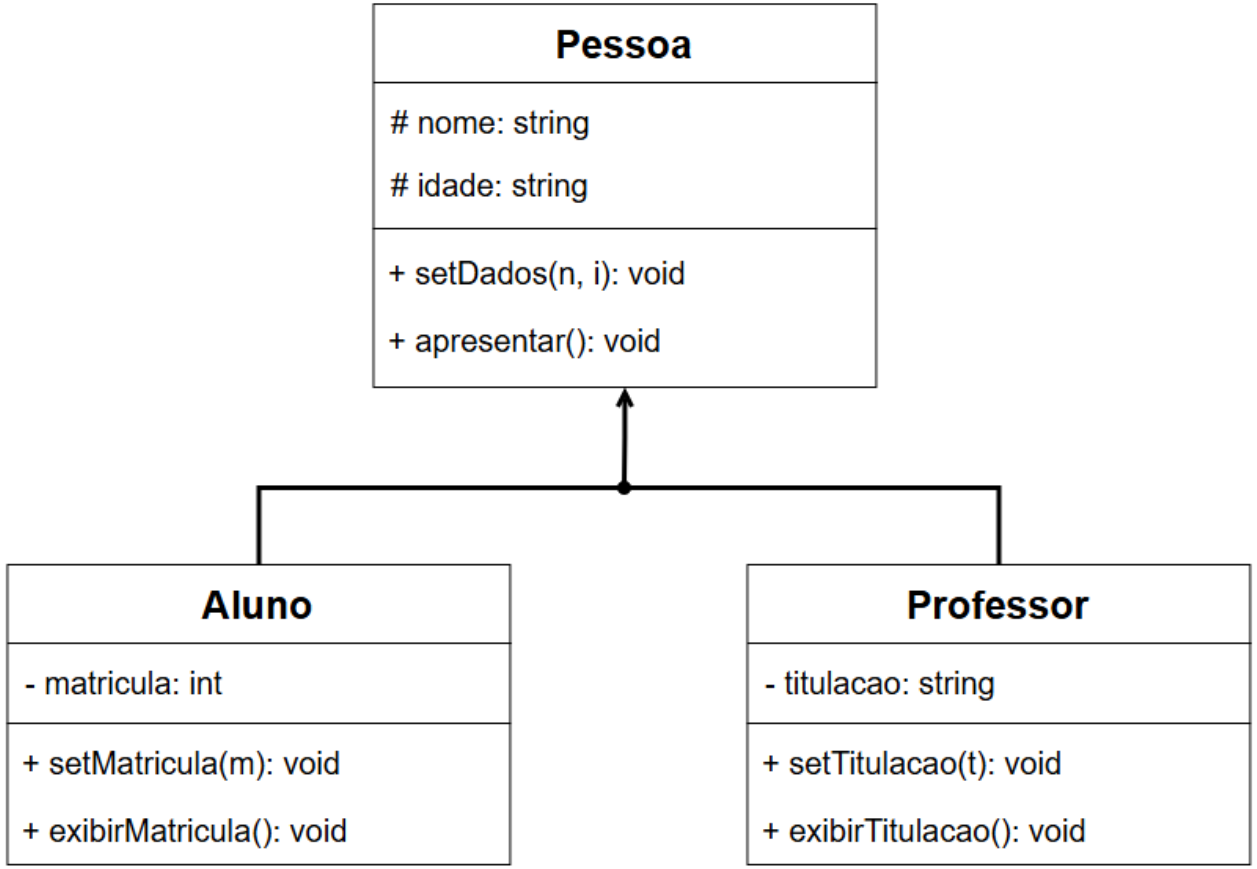
Exemplo

```
class Pessoa {
protected:
string nome;
int idade;
public:
void setDados(string n, int i){ nome = n; idade = i; }
void apresentar() const {
      cout << “Nome: “ << nome << “, idade: ” << idade << endl;
};
class Aluno : public Pessoa {
private:
int matricula;
public:
void setMatricula(int m) const { matricula = m;}
void exibirMatricula() const {
      cout << “Matricula: “ << matricula << endl; }
};
```

```
class Professor : public Pessoa {
private:
string titulacao;
public:
void setTitulacao(int t) const { titulacao = t;}
void exibirTitulacao() const {
      cout << “Matricula: “ << matricula << endl; }
};

int main() {
    Aluno a;
    a.setDados("Maria”, 17); a.setMatricula(2026001);
    a.apresentar();
    a.exibirMatricula();
    Professor p;
    p.setDados(“Fulano”, 30); p.setTitulacao(“Doutor”);
    p.apresentar();
    return 0;
}
```

Diagrama UML com herança de classes



Exercícios

Exercício 1

Crie uma classe base Veiculo com os atributos `marca()` e `ano()`, um método `setDados()` e um `exibir()` que imprime os dois. Crie duas classes derivadas por herança pública, Carro, com o atributo privado `portas ()` e métodos para defini-lo e exibi-lo, e `Moto`, com o atributo privado `cilindrada()` e métodos equivalentes. No `main`, crie um carro e uma mota, use os métodos herdados e os próprios de cada um, e mostre que `exibir()` funciona nos dois sem ter sido escrito em nenhuma das derivadas. Faça um diagrama UML da hierarquia deste problema.

Exercício 2

Escolha um domínio à sua escolha, como transporte, jogos, música ou animais, e proponha uma hierarquia com uma classe base e duas derivadas. Em texto, liste quais atributos e métodos ficariam na base e por quê, e o que seria específico de cada derivada. Justifique por que a relação escolhida é herança e não composição.